Assignment 1

Tutors: Ziming Hong, Li He, Yuhao Wu
Group members: Nathan Roland - 520510542, nrol3229; Leheng Li - 500175118, leli6236; Matthew Xu - 510441526, maxu0028; Henry Nguyen - 510581419, mngu3048

Abstract

This report discusses the robustness of Non-negative Matrix Factorisation (NMF) algorithms against image datasets corrupted with high-magnitude noise. NMF is a powerful unsupervised learning technique that factorises a non-negative data matrix into two lower-dimensional non-negative matrices, yielding part-based and interpretable representations. The standard NMF algorithm is vulnerable to outliers and noise, which undermines its performance on real data. To address this issue, we implement and compare three NMF algorithms: Classical NMF, L1-Norm Based NMF, L2,1-Norm Based NMF; with Hypersurface Cost NMF also implemented but was not used for evaluation. We assess each algorithm's resilience to various noises, particularly varying degrees of salt and pepper noise, using the ORL and Extended YaleB datasets.

The performance of each method is evaluated using Relative Reconstruction Error (RRE), Average Accuracy, and Normalised Mutual Information (NMI).

1 Introduction

Non-negative Matrix Factorisation (NMF) is a matrix factorisation technique used in unsupervised learning. It decomposes a given non-negative matrix into two lower-dimensional non-negative matrices, making data easier to interpret while preserving positivity.

The non-negativity constraint makes the learned features additive and interpretable, and NMF is particularly favorable for image analysis applications, document classification applications, and bio informatics applications. The classical NMF minimizes the Frobenius norm of the reconstruction error and can be solved efficiently using multiplicative update rules, and it is a straightforward but powerful method for disclosing the hidden data patterns. However, in practice, datasets will include outliers and noise that will severely degrade the performance of standard NMF. When data are tainted, NMF's squared-error loss can cause large residuals to dominate the loss function, leading to poor factorisations as well as reduced interpretability. To compensate for this drawback, several robust NMF variants have been proposed. The L1-Norm Based NMF replaces the L2 reconstruction term with an L1 term, which is less outlier-sensitive and sparsity-inducing. The L2,1-Norm Based NMF incorporates further robustness through the application of the L2,1 norm to multi-dimensional noise and providing higher generalisation when acting in high-dimensional spaces. The Huber-Cauchy (HC) Loss Based NMF incorporates a differentiable and robust loss function that limits the influence of outlier-inducing residuals but allows smooth optimisation dynamics.

This problem aims to utilize and compare the stability of these NMF algorithms on two face image databases in real-world scenarios: the ORL database and the Extended YaleB database. To simulate real-world noise, we introduce salt-and-pepper noise with different probability and ratio parameters ((p, r)), which corrupts images with white (salt) and black (pepper) pixels. To see how well an algorithm can turn noisy data into clear images, it is tested at different levels of noise. We use three quantitative measurement metrics to check the quality of reconstruction and the performance of clustering: Relative Reconstruction Error (RRE), Average Accuracy, and Normalised Mutual Information (NMI). This report aims to identify the most resilient NMF algorithm against noise through these experiments, thereby providing insights into their suitability for real-world data analysis applications in noisy or corrupted environments.

2 Related Work

2.1 Classic NMF

As mentioned, Non-negative Matrix Factorisation (NMF) is a matrix factorisation technique used in unsupervised learning. It decomposes a given non-negative matrix into two lower-dimensional non-negative matrices, making data easier to interpret while preserving positivity[16]. The classic NMF algorithm utilises the objective function of:

$$\min_{\substack{D\geq 0\\ R\geq 0}} \|X - DR\|_F^2$$

where:

- $X \in \mathbb{R}^{d\times n}$ is the input data matrix
- $D \in \mathbb{R}^{d\times k}$ is the dictionary (or basis) matrix
- $R \in \mathbb{R}^{k\times n}$ is the coefficient (or encoding) matrix

and all entries of D and R are constrained to be non-negative. X, D and R can be swapped with V, W and H in order.

The use of the classic NMF provides various advantages in not only data handling, but also real world use. The non-negativity constraint not only leads to more interpretable factors but also guarantees convergence to a local minimum. Another advantage of the algorithm is that it does not require manual step size tuning, which simplifies its practical use. NMF is particularly useful for analysing images, text prediction, and pattern recognition.

However, there are some obvious disadvantages of the classical NMF. Classical NMFs are sensitive to outliers and noisy data, and thus experience inaccuracies for real world data where corruption is present. In conjunction, there is the possibility of overfitting with the NMF learning noise patterns instead of the structure, thus resulting in inaccurate data. Finally, there is non-uniqueness present in the classical NMF, meaning that multiple pairs of D and R can predict X. This results in confusion over whether the factorisation represents the underlying structure, and thus it is difficult to draw conclusions over the data.

To address these concerns, models such L1-Norm Based Algorithm, L2,1 - Norm Based Algorithm and Huber-Cauchy loss algorithm have been utilised.

2.2 L1 - Norm Based NMF

The L1-Norm Based Algorithm is designed to help reduce large additive noise without needing to know the locations. The use of the L1 Norm (Manhattan Distance) enhances the robustness to noise and outliers in the data, through the addition of a large additive noise matrix. It models the partial corruption, which is treated as large additive noise and it is

able to simultaneously learn the basis matrix, coefficient matrix and estimate the positions and values of noise in the data. The objective function of the L1 Norm Based Algorithm is displayed:

$$\min_{W,H} \|V - WH\|_1 = \sum_{i,j} |V_{ij} - (WH)_{ij}|$$

$$\text{subject to} \quad W \geq 0,\ H \geq 0$$

where:

- V : input matrix [m× n]
- W : basis matrix [m× k](Non negative)
- H : coefficient matrix [k× n](Non negative)
- i, j: random row and column values inside matrix

The purpose of the objective function is to guide the algorithm in the ideal direction, minimising reconstruction errors, and imposing penalties to increase the sparsity of the matrix to replicate real life examples.

The advantage of utilising the L1- Norm Based Algorithms include the sparsity, and robustness to outliers, making it more useful in real world applications[1]. Since the L1-Norm Based NMF imposes penalties to increase the sparsity of the matrix, it allows for minimization of the number of features, making it useful when analysing real world data. In conjunction, the L1- Norm Based Algorithm results in more robust values when using outliers, since the outliers aren't squared, thus it is less affected. This results in the L1 - Norm Based algorithm being more desirable to utilise on real world data, where extreme outliers may occur and the algorithm can focus on small amounts of active features. Examples of real world use of L1 Norm Based Algorithms include image processing, bio-informatics, and feature selection[2].

However, there are disadvantages involved with the L1- Norm Based NMF. L1- Norm is not differentiable at zero, which complicates the given optimisation process (which will be addressed in the methods). In conjunction, it is possible that multiple solutions can yield the same L1 norm, thus not producing unique results. This ambiguity may reduce stability of the resulting factors. Therefore, because L1 focuses on absolute deviations, it may result in a loss of reconstruction accuracy compared to a L2,1 Norm Based Algorithm, where the data is relatively clean.

2.3 L2,1 - Norm Based NMF

The L2,1 Norm Based NMF has been established as an efficient and robust method to apply to domains sensitive to outliers, as well as a method to iteratively update and approximate the L2 objective function[16]. The computation of error for each data point as a non-square value results in outliers not dominating the objective function, thus reducing noise and outliers and resulting in a sparse solution. The objective function can be classified as:

$$\min_{A,Y} \|X - AY\|_{2,1} + \lambda \|Y\|_{2,1}$$

where:

- X is the input data matrix
- A is the basis matrix
- Y is the coefficient matrix
- λ is the regularisation parameter controlling the trade-off between reconstruction error and sparsity

The key defining feature that separates the L2,1 Norm Based NMF from the L1 Norm Based NMF is the fact that L2,1 is based upon rows, rather than L1 which is more individual value

based. L2,1 encourages row-wise sparsity, checking if an entire row has large errors/corruption, then the whole row gets small weigths. Therefore if a row is heavily corrupted, the L2,1 Norm Based NMF prevetns it from dominating the factorisation, and thus cleaner rows have more of an influence in learning. Therefore it is more robust, as the row wise norm groups correlated corruptions together.

Advantages of utilising a L2,1 Norm Based Algorithm stem from both the advantages of L2 and L1 Norm Based Algorithms. The use of the L2,1 norm on the residual error matrix ensures that the NMF is robust to high dimensional data, and effective in dealing with complex domains. This makes it extremely useful with real world applications such as gene selection and feature extraction[3].

However, there are drawbacks which include the over reliance on the regularisation parameter and the difficulty of fine tuning the value to achieve the best possible performance. Another issue is the non-smoothness obtained through the cost function, thus resulting in the multiplicative update rules not applying to the NMF. Finally, there is potential that in some datasets, the L2,1 Norm Based NMF can be less effective for sparse noise and outliers than the L1 Norm based NMF.

2.4 Huber-Cauchy (HC)

The algorithm based on Huber-Cauchy (HC) loss is a method of non-negative matrix factorisation (NMF) designed to address the critical issue of inherent sensitivity to outliers in traditional squared error approaches. It achieves this by employing an objective function specifically engineered for robust handling of large residuals. The objective function for the HC loss-based algorithm is as follows:

$$\min_{W \geq 0,\, H \geq 0} \sum_{i,j} \frac{1 + (V_{ij} - (WH)_{ij})^2}{(V_{ij} - (WH)_{ij})^2}$$

where V is the input matrix, W the basis matrix, and H the coefficient matrix, and W and H are both non-negative. The above objective function guides the algorithm towards optimal direction in minimizing reconstruction error and imposing some limitations on the effect of outlier residuals, hence replicating robust behavior in actual applications.

The merits of employing the HC loss-based algorithm are its resistance to outliers, thereby optimizing practical application.The architecture of the loss function guarantees that the influence of outliers on the overall error plateaus as their intensity escalates, thereby preventing them from overshadowing the decomposition process. Also, the HC loss-based approach is always differentiable, and this offers a significant practical advantage over using L1 norm. Smoothness enables direct utilization of highly efficient, well-established gradient descent optimization methods with more stable and faster convergence compared to those requiring subgradient methods. The HC loss-based approach is therefore better suited for real data with extreme outliers where there is a need for smooth, robust optimisation path. Practical applications of HC loss and other robust NMF techniques include image processing, bioinformatics, and recommendation systems.

However, HC-loss-based NMF also possesses drawbacks. Whilst differentiable, the complex nonlinear nature of the gradient (proportional to (1+r2)2r) may render the optimisation process sensitive, necessitating particularly careful tuning of the learning rate to prevent instability or convergence to poor local minima. Furthermore, unlike L1 norm, the HC loss itself does not inherently promote sparsity in the factor matrices. Consequently, if feature selection or extreme piecewise sparsity is the primary objective, additional penalty terms must be incorporated. Furthermore, as HC focuses on limiting the impact of large deviations, it may result in some loss of reconstruction accuracy compared to standard L2 Frobenius norm-based algorithms when applied to relatively clean data.

3 Methods

3.1 Data Preprocessing

To test the NMFs, we utilised the ORL dataset and the extended YaleB dataset. The ORL[7] dataset is the set of face images taken between April 1992 and 1994, used in a face recognition project carried out by Cambridge University. There are 10 different images of 40 distinct people, all with different times, lighting, facial expressions, and facial details (glasses/no glasses). All images were taken with the subjects in a frontal, upright position with a dark homogeneous background.The size of each image is 92 x 112 pixels, with 256 grey levels per pixel.

The extended YaleB[10] dataset contains 2414 images of 38 individuals, created by the researchers at Yale University under various laboratory controlled lighting conditions. They have 9 poses and 64 illumination conditions, which helped in doppelganger list comparison and facial recognition projects. All images are resized to 168 x 192 pixels after being manually aligned and cropped.

In order to perform effective salt and pepper noise testing (explained in section 3.6), we have to normalise both datasets into 0-255 values. This is to ensure that proper salt and pepper values are created, and thus the robustness of the NMFs can be tested.

3.2 Classical NMF

The classical NMF function utilises the objective function of:

$$\min_{\substack{D\geq 0\\ R\geq 0}} \|X - DR\|_F^2$$

where X is the input data matrix (d x n), D is the dictionary matrix (d x k), R is the coefficient matrix (k x n), and D and R both have all positive inputs. The optimisation method utilised is the Multiplicative Update Rules, which aims to scale each element multiplicatively to perform gradient descent, instead of using additive steps which could cause negative values. The multiplicative update rules are derived from the gradient of the objective function, thus ensuring non negative values, and a convergence to local minimum[8]. To implement the Basic NMF, we utilise the following four functions: intialise_matrices, updateR, updateD, and nmf_fit.

The function **initialise_matricies** sets up the dictionary matrix and coefficent matrix with random positive values, multiplying them with a scale factor to ensure they are appropriate. The functions **updateR** and **updateD** update the dictionary and coefficient matrix using multiplicative update rules, acquiring the numerator (correlation between basis vectors and data) and denominator (current reconstruction projected), adding a small value to each (to prevent division by zero) and then multiplying each element of the given matrix by the new numerator over the new denominator to update the values. Finally, the **nmf_fit** function adds all the functions together, intialising the matricies based on the given data (X) and number of factors (n_components). Then, for each iteration, the R and D matricies are updated and this is continued until all iterations have passed or if the NMF has converged.

While the classical NMF is easy to implement, from a theoretical point of view, it is not robust in dealing with outliers. This is due to the Forbenius Norm as the cost funciton, where error is constantly squared, thus outlier values are maximised compared to other NMFs. This means that the classical NMF would have extremely poor performance with any data with extreme outliers/noise (especially salt and pepper).

3.3 L1 Norm Based NMF Implementation

The objective function of the L1 Norm Based Algorithm is displayed:

$$\min_{W,\,H} \|V - WH\|_1 = \sum_{i,j} |V_{ij} - (WH)_{ij}|$$

$$\text{subject to} \quad W \geq 0,\ H \geq 0$$

where:

- V : input matrix [m× n]
- W : basis matrix [m× k](Non negative)
- H : coefficient matrix [k× n](Non negative)
- i, j: random row and column values inside matrix

In order to optimize the algorithm, the optimization methods of majorisation-minimisation algorithms and a modified multiplicative rules update. Majorisation-minimisation algorithms consist of utilising a locally tight upper-bound (majorising), and then successively minimizing these upper-bounds in order to reach a convergence point[11]. The majorisation step involves utilising the least squares method to gain weights for matrix values[12], which can be seen in the following equation:

$$w_{ij} = \frac{1}{\sqrt{(V_{ij} - (WH)_{ij})^2 + \varepsilon^2}}$$

The minimisation step is then performed in conjunction with the modified multiplicative rules[9] update, as the numerator and denominator are both affected by the weight value, and the basis and coefficient matriticies are updated using the following equations:

$$H \leftarrow H \odot \frac{W^T\,(W_{\text{weights}} \odot V)}{W^T\,(W_{\text{weights}} \odot (WH))} \tag{1}$$

$$W \leftarrow W \odot \frac{(W_{\text{weights}} \odot V)H^T}{(W_{\text{weights}} \odot (WH))H^T} \tag{2}$$

To implement the L1 Norm Based NMF, we utilise the following 4 functions: l1_initialise_matricies, l1_update_H, l1_update_W, and l1_nmf_fit.

Similar to the Classical NMF initialise function, the **l1_initialise_matricies** function intialises the bases and coefficient matricies with random values based upon the scale factor which is determined by the input matrix. The **l1_update_H** and **l1_update_W** functions both apply the Majorisation-Minimisation algorithms, computing weights for l1 approximation, finding the residual values, and then utilising the mutliplicative update rules to obtain the numerator and denominator, via the weights calculated. Small values are then added onto each of the denominator and numerator, and the basis and coefficient matrix values are updated by multiplying the new numerator over the new denominator. Finally, the **l1_nmf_fit** function combines all these functions together, intialising W and H with random non negative numbers, and for each iteration it computes the residual and weights from the residual, updating both W and H until the function converges to a local minimum or until all iterations are performed.

In theory, the L1 Norm based NMF should be extremely robust in regards to outliers. The use of a majorisation-minimisation optimisation method means that any outlier elements get larger residuals, and in return get smaller weights individually, thus more robustness to noise and outliers. In conjunction, the non negativity is preserved, and same with any multiplication by zero errors. This ensures that the NMF can run smoothly and accurately, thus producing accurate data and accurate reconstructions.

3.4 L2,1 Norm Based NMF Implementation

The objective function of the L2,1 Norm Based NMF Implementation can be classified as:

$$\min_{A,Y} \|X - AY\|_{2,1} + \lambda \|Y\|_{2,1}$$

where:

- X is the input data matrix
- A is the basis matrix
- Y is the coefficient matrix
- λ is the regulisation parameter controlling the trade-off between reconstruction error and sparsity

The optimisation function is also a maximisation-minimisation, but instead of doing each element individually, the whole row is weighted at once, thus minimising corrupted rows. We are minimising the reconstruction error through calculation of the residual, and from there, row weight calculation. The row weights are represented by the following equation: $w_i = \frac{1}{\|R_i\|_2}$ From this point, we then apply the weight to the whole role, thus minimising corrupted rows and exemplifying "pure" rows, therefore getting more accurate training data. To implement the NMF, the following functions are called: l2_initialise_matricies, l2_norm, l2_compute_weights, l2_update_H, l2_update_W, and l2_nmf_fit.

The function **l2_initialise_matricies** initialises the basis and coefficient matricies with random data values, multiplied by a scale factor determined by the input matrix. The **l2_norm** function performs the L2 normalisation of each row and then squares the value, returning the sum of all l2 row norms to gain the l2,1 norm. The **l2_compute_weights** function performs the majorisation optimisation method by computing the row weights based on the row residual, ensuring that all elements in the row share the same weight. Both the **l2_update_H** and the **l2_update_W** functions are in charge of minimisation optimisation methods, using the weights calculated from the residuals to update the relevant matricies. Finally, the function **l2_nmf_fit** combines all the functions together, initialising the matricies, computing the residual, new values and cost for each iteration, and stopping once it reaches a convergence point.

In theory, the L2,1 Norm Based NMF should produce a much more robust functionality than the L1 Norm based NMF and the classical NMF. This is due to handling and lowering weights of rows with more corruption[13], thus they have less effect on the training data. The combination of using L2 and L1 methods means that the factorisation is balanced with element-wise robustness and row-wise coherence. Therefore, the L2,1 Norm Based NMF should be perfectly equipped to handle any noise corruption in the dataset.

3.5 Huber-Cauchy Implementation

As mentioned, the objective function for the HC loss-based algorithm is as follows:

$$\min_{W \geq 0,\, H \geq 0} \sum_{i,j} \frac{1 + (V_{ij} - (WH)_{ij})^2}{(V_{ij} - (WH)_{ij})^2}$$

where V is the input matrix, W the basis matrix, and H the coefficient matrix, and W and H are both non-negative.

To optimise the objective,We apply a gradient descent-based approach to iteratively update W and H as follows:

$$W \leftarrow \max\left(0, W - \eta \nabla_W L\right), \tag{3}$$

$$H \leftarrow \max\left(0, H - \eta \nabla_H L\right), \tag{4}$$

where η is the learning rate and the gradients are given by:

$$\nabla_W L = -4\left(\frac{V - WH}{(1 + (V - WH)^2)^2}\right) H^T, \tag{5}$$

$$\nabla_H L = -4W^T\left(\frac{V - WH}{(1 + (V - WH)^2)^2}\right). \tag{6}$$

After each update, W and H are normalised column-wise to avoid numerical instability:

$$W \leftarrow \frac{W}{\|W\|_2}, \quad H \leftarrow \frac{H}{\|H\|_2}. \tag{7}$$

The algorithm terminates when the change in the loss value between iterations falls below a predefined threshold ε, ensuring convergence.

Theoretically, from the point of view of loss function design, the HC loss has the benefit of both smooth differentiability and bounded influence. It behaves like the squared loss for small residuals (enabling accurate local fitting), and for large residuals, the loss saturates—avoiding domination of the reconstruction by outlying pixels (e.g., the 0 and 255 values for salt-and-pepper noise). This hybrid property gives robustness together with effective optimisation, without L1-type function instability but with noise resilience preserved.

3.6 Salt and Pepper Noise

To test the effectiveness of the NMFs and their ability to reconstruct images, salt and pepper noise with different settings is utilised. Salt and pepper noise corrupts pixel values in the image, where the maximum possible value is 255 (salt noise) and the minimum possible value is 0 (pepper noise)[4]. An example of salt and pepper noise corruption can be seen below:

$$4 \times 4 \text{ original image}: \begin{bmatrix} 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 \end{bmatrix}$$

$$\text{image by impulse noise}: \begin{bmatrix} 100 & 255 & 100 & 100 \\ 100 & 255 & 100 & 100 \\ 255 & 100 & 100 & 0 \\ 100 & 100 & 100 & 100 \end{bmatrix}$$

Figure 1: Example of salt and pepper noise in a 4 x 4 matrix, with (0.25, 0.75) probability parameters

To control the salt and pepper noise in given data, we utilise two probability parameters: p and r. The total portion of corrupted pixels is given by p, and the salt vs pepper noise ratio is given by r. For example, the probability parameters (p, r) of (0.3, 0.8) would result in 30% of the image pixels being corrupted, with 80% of those corrupted pixels having a value of 255 (salt) and 20% having a value of 0 (pepper). In order to properly test the NMFs, we utilised the following 4 pairs of (p, r): (0.2, 0.1), (0.2, 0.7), (0.4, 0.1) and (0.4,

0.7). This helps us understand how robust the NMF is when noise intensity increases, as well as if it can handle both salt corruption and pepper corruption, revealing any biases[5].

Salt and pepper noise testing is extremely useful for evaluating the NMFs due to their ability to test robustness, real world likeness, and clear visual evaluation/difference. Salt and pepper noise creates extreme outliers in the data, setting the values to either 0 or 255. This allows the robustness of the algorithms to be demonstrated, with NMFs that magnify outlier influence performing worse than NMFs which aim to minimise outlier influence. In conjunction, salt and pepper noise replicates real world scenarios, thus determining whether a NMF can be useful in a non controlled environment. The noise is usually generated by a malfunctioning of pixel elements in camera sensors, errors in conversion process, or lack of storage space, and therefore, testing a NMF in environments where they deal with salt and pepper noise can be helpful in determining which models can separate structures from sparse corruption[6]. Finally, it is easier to highlight the difference between the norms with the classical NMF, L1 Norm based NMF, and L2 Norm based NMF all dealing with salt and pepper noise in different ways, thus illustrating the key differences between these factorisations.

3.7 Other types of noise

While we have implemented other nosies in the code (Gaussian and Uniform noise), we decided to mainly focus on salt and pepper noise due to the fact that we can easily highlight the effectiveness of each NMF algorithm more clearly. Salt and pepper noise creates extreme outliers, which clearly highlight the robustness of the NMFs. We can see the advantages and disadvantages of NMFs clearly, as well as which pixels were effected. In conjunction, salt and pepper is more reflective of real world faults, thus it should be utilised in the experiment.

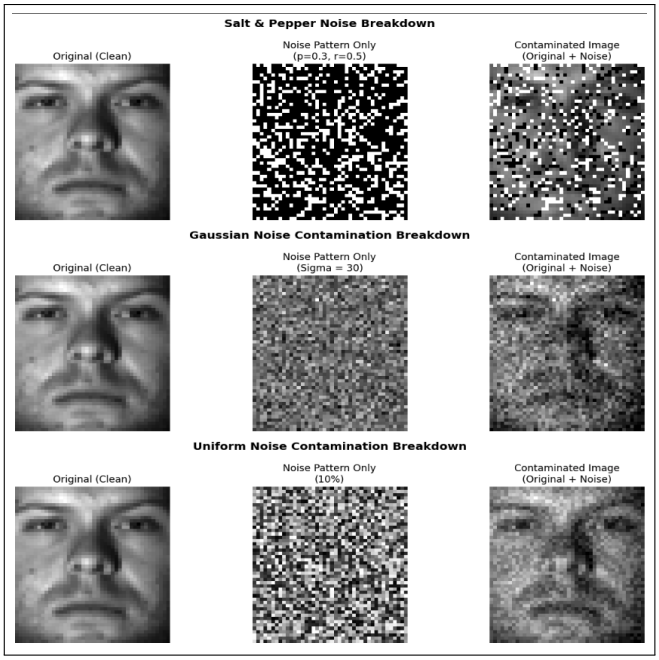

Figure 2: Examples of all the noises applied to the Yale B dataset

Gaussian noise is a statistical noise characterised by the probability density function of the normal distribution. It is a noise generated by adding random values that are normally distributed with a mean of zero and a standard deviation to the input data. Uniform noise is where every value in a range has equal probability, and is useful for simulations requiring flat randomness (number generation)[15]. However, both gaussian and uniform noise share the same trait that it is dense and mild, with all pixels effected. Therefore, the classical NMF can deal with the noise corruption with relative ease, and thus the differences between

the classical NMF and the other NMFs aren't highlighted. In conjunction, these noises do not have extreme outliers, meaning that the L2,1 and L1 robustness isn't really tested, and therefore, it would be less informative for our project to utilise these noises instead of salt and pepper noise.

4 Experiment

One of the goals for this study was to assess and compare the reconstruction performance of different NMF algorithms when applied to contaminated (noisy) image data. For this reason, we compare our implemented NMF algorithms (Multiplicative Update, L1-norm based, and L2,1-norm based) in a set of experiments designed to evaluate the performance of each algorithm on different levels and patterns of noise.

Salt and pepper noise was chosen for this assessment as it is commonly observed in real-world image data, particularly in images captured in low-light environments by weaker (usually film) camera technologies and digital images that have experienced some form of degradation during transmission [4]. In order to control the amount of noise and the distribution of noisy black-white pixels in the contaminated image, we introduce the two parameters p - the proportion of noisy pixels, and r - the proportion of white/black noisy pixels. In our assessment, each algorithm was tested against four noise settings:

- p = 0.2, r = 0.1 : simulating a moderate amount of noise with primarily black noisy pixels
- p = 0.2, r = 0.7 : simulating a moderate amount of noise with primarily white noisy pixels
- p = 0.4, r = 0.1 : simulating a large amount of noise with primarily black noisy pixels
- p = 0.4, r = 0.7 : simulating a large amount of noise with primarily white noisy pixels

To quantify the reconstruction performance of these algorithms, we chose three metrics: Relative Reconstruction Error (RRE), Average Accuracy, and Normalised Mutual Information (NMI). These three metrics are all measures of similarity, where RRE measures the magnitude of errors in a reconstructed output relative to the size of original data, and average accuracy and NMI measure clustering performance between the reconstructed noisy images and the original data. Strong clustering performance between reconstructed and original images is an indicator of superior reconstruction accuracy, so average accuracy and NMI scores on clusters can both be applied to assess the performance of our NMF algorithms.

To improve the robustness of our results, experiments are repeated 5 times, with each run randomly sampling 90% of the dataset. The metrics are recorded for each run, then reported as the average of 5 runs in the tables below. Experiments are also repeated with identical parameters on both the ORL and extended YaleB image datasets, exposing some differences in algorithm performance when applied to different datasets.

4.1 ORL results

Table 1: Performance of different NMF algorithms on ORL dataset with varying salt-and-pepper noise levels (p, r).

Algorithm	(p, r)	**RRE (s.d.)**	**Average Accuracy (s.d.)**	**NMI (s.d.)**
Multiplicative update	(0.2, 0.1)	0.2701 (0.0005)	0.5494 (0.0120)	0.7281 (0.0090)
	(0.2, 0.7)	0.2575 (0.0009)	0.5356 (0.0242)	0.7027 (0.0132)
	(0.4, 0.1)	0.4119 (0.0001)	0.3639 (0.0084)	0.5515 (0.0127)
	(0.4, 0.7)	0.3724 (0.0014)	0.3244 (0.0143)	0.5115 (0.0099)
L1-norm based	(0.2, 0.1)	0.3141 (0.0012)	0.1978 (0.0011)	0.3894 (0.0098)
	(0.2, 0.7)	0.3064 (0.0005)	0.1967 (0.0092)	0.3827 (0.0161)
	(0.4, 0.1)	0.3853 (0.0014)	0.2000 (0.0039)	0.3836 (0.0075)
	(0.4, 0.7)	0.3258 (0.0008)	0.2000 (0.0039)	0.3770 (0.0198)
L2,1-norm based	(0.2, 0.1)	0.2682 (0.0006)	0.5794 (0.0100)	0.7462 (0.0076)
	(0.2, 0.7)	0.2561 (0.0009)	0.5661 (0.0315)	0.5661 (0.0315)
	(0.4, 0.1)	0.4121 (0.0003)	0.3789 (0.0109)	0.5680 (0.0170)
	(0.4, 0.7)	0.3730 (0.0014)	0.3278 (0.0177)	0.5081 (0.0189)

4.2 YaleB results

Table 2: Performance of different NMF algorithms on the ORL dataset under varying salt-and-pepper noise levels (p, r).

Algorithm	(p, r)	**RRE (s.d.)**	**Average Accuracy (s.d.)**	**NMI (s.d.)**
Multiplicative update	(0.2, 0.1)	0.2686 (0.0008)	0.1996 (0.0073)	0.2709 (0.0058)
	(0.2, 0.7)	0.3663 (0.0013)	0.1614 (0.0087)	0.2213 (0.0150)
	(0.4, 0.1)	0.4010 (0.0006)	0.1518 (0.0082)	0.1917 (0.0110)
	(0.4, 0.7)	0.6208 (0.0021)	0.1095 (0.0048)	0.1221 (0.0077)
L1-norm based	(0.2, 0.1)	0.3376 (0.0032)	0.0834 (0.0020)	0.0854 (0.0018)
	(0.2, 0.7)	0.3134 (0.0021)	0.0826 (0.0010)	0.0827 (0.0028)
	(0.4, 0.1)	0.4414 (0.0092)	0.0834 (0.0025)	0.0846 (0.0049)
	(0.4, 0.7)	0.3381 (0.0033)	0.0861 (0.0031)	0.0961 (0.0054)
L2,1-norm based	(0.2, 0.1)	0.2681 (0.0007)	0.2027 (0.0073)	0.2714 (0.0053)
	(0.2, 0.7)	0.3664 (0.0013)	0.1716 (0.0059)	0.2242 (0.0122)
	(0.4, 0.1)	0.4002 (0.0006)	0.1580 (0.0093)	0.1983 (0.0081)
	(0.4, 0.7)	0.6207 (0.0022)	0.1090 (0.0075)	0.1256 (0.0048)

4.3 Discussion

Recall the nature of the three metrics selected - RRE measures the relative error between a reconstructed image and its original counterpart, with values closer to 0 indicating a smaller error and values closer to 1 indicating a larger error. Average accuracy and NMI measure clustering performance, with values closer to 1 indicating that the majority of reconstructed images are similar enough to their original counterparts to be placed in the same clusters, whereas values closer to 0 indicating that reconstructed images either do

not exhibit significant visual similarity to the originals, or even that reconstructed images exhibit visual features that may cause them to be clustered into an entirely different class.

Observing the RRE of all three assessed algorithms on the ORL dataset, we find that both the proportion of noisy pixels in the image and the distribution of white/black noise significantly impacts the RRE of each algorithm. Across all algorithms, a larger p value (more noisy pixels) universally increased the RRE of reconstructed images, signifying that increasing the amount of noise tends to lower the reconstruction performance of all the NMF algorithms tested. On the ORL dataset, an increase from 20% noisy pixels to 40% with all other variables held constant resulted in an average increase in RRE of 0.1014. This trend was observed to an even greater extent on the YaleB dataset, where an increase from 20% to 40% noisy pixels resulted in an average increase in RRE of 0.1503. The greater impact of increased noise on the YaleB dataset can potentially be attributed to the larger dimensions of the image data in the YaleB dataset, with images in the YaleB dataset containing 32256 pixels compared to 10304 pixels in the ORL dataset, meaning each NMF algorithm is required to reconstruct more pixels whilst also dealing with a larger magnitude (but not proportion) of noise.

It is also evident that the distribution of white and black noise contributes to the average RRE of each algorithm, though to a lesser extent than the proportion of noise. The exact effect of changing the white/black noise distribution also seems to be dependent on both the specific algorithm being used and the characteristics of the underlying dataset - unlike increasing the proportion of noise which invariably increases RRE across all algorithms. Consider the results for all algorithms on the ORL dataset, where changing the value of r from 0.1 to 0.7 universally decreased RRE across all algorithms. Notably, this increase was more pronounced when the proportion of noise was higher, with an average change of -0.1 when p = 0.2 and -0.4 when p = 0.4. However, when repeating the experiment on the YaleB dataset, increasing the value of r from 0.1 to 0.7 significantly increased RRE for the Multiplicative Update and L2,1-norm algorithms, whereas the L1-norm algorithm retained the same trend. Our hypothesis is that this irregularity in the results can be explained in part by underlying differences in the characteristics of both datasets - e.g. there are more bright/white pixels in the YaleB dataset compared to the ORL dataset, so increasing the proportion of black noisy pixels is far more impactful on the YaleB set, resulting in a disproportionate increase in RRE. This presents an opportunity for further investigation that was not pursued in this study.

Differences in dataset characteristics do not explain the fact that the L1-norm algorithm exhibited a similar response to increased black noisy pixels across both datasets - suggesting that to some extent that the mechanisms behind algorithms can impact sensitivity to white/black noise in image data. This is supported by inspecting the SD of RRE figures across both datasets when moving between r = 0.1 and r = 0.7, where it becomes evident that the distribution of white/black noisy pixels has a significant impact on the stability of these algorithms. For example, it seems that both Multiplicative Update and L2,1-norm algorithms are more robust and stable when handling against lighter-coloured noise, though once again this result may be skewed by the underlying characteristics of pixel colour distribution in the datasets.

We now move on to consider the average accuracy metric. Upon inspecting the figures above, it becomes immediately evident that the Multiplicative Update and L2,1-norm algorithms display extremely similar performance across both datasets - a trend that may have been noticed by astute observers while inspecting the results for RRE. This trend is also mirrored in the NMI metric, suggesting that our implementations of these two algorithms may have converged upon a similar local optimum despite significantly different objective functions. We observe that increasing the proportion of total noise from 20% to 40% results in a decrease in reconstruction performance for the Multiplicative Update and L2,1-norm algorithms, decreasing average clustering accuracy by anywhere between 20% on the ORL dataset to 5% on the YaleB dataset. Once again, the L1-norm algorithm displays a unique set of results, displaying remarkable invariance in average clustering accuracy across both datasets despite significant changes in the amount of noise. Moreover, whilst increasing the proportion of black noise results in a clear change in average clustering accuracy for

the Multiplicative Update and L2,1-norm algorithms (typically decreasing accuracy), the L1-norm algorithm once again remained largely unaffected.

Another key observation can be made regarding the relationship between RRE and average clustering accuracy. In particular, a decrease in RRE should theoretically result in an increase in average clustering accuracy, considering that fewer errors should indicate that the reconstructed image is visually more similar to the original. Whilst this logic is supported by some examples (see Multiplicative Update and L2,1-norm on the YaleB dataset), it is also contradicted by others (e.g. Multiplicative Update on the ORL dataset). This indicates that the relationship between RRE and clustering accuracy is perhaps quite complex, potentially owing to the mechanisms for the clustering algorithms themselves.

Finally, we move on to inspect the results for our NMI metric across both datasets. Considering that NMI and average accuracy are both metrics that assess the clustering performance of reconstructed images versus their original counterparts, we expect to see some similarities in the results for these two metrics. Immediately upon viewing the results for the ORL dataset it becomes evident that the NMI figures closely follow the trend, but not magnitude of the results for average accuracy. Across all three algorithms, a relative increase in average accuracy is reflected by a relative increase in NMI, which is an expected outcome. This is also true of the YaleB dataset. We were unable to identify any significant trends in the NMI results outside of those already identified in the results for average accuracy.

From a wider perspective spanning performance on both datasets, we find that RRE remained fairly consistent for each algorithm despite changes in the dataset, indicating that reconstruction error is at least somewhat independent from the size of the input data. This is not the case for clustering metrics, however, as all three algorithms displayed significantly better clustering accuracy and NMI on the ORL dataset (smaller image size) compared to the YaleB dataset. While this was identified earlier on in our discussion, the sheer magnitude of the performance difference between the two datasets (with algorithms performing 2-3x better on clustering metrics on ORL compared to YaleB) indicates that the more data an NMF algorithm is expected to decompose and reconstruct, the worse the overall reconstructed product.

5 Conclusion

This paper experimented with the stability of three NMF algorithms — Multiplicative Update, L2,1-norm, and L1-norm — on noisy image datasets (ORL and YaleB) using RRE, average clustering accuracy, and NMI. Results showed that the ratio increase of noise consistently raised RRE and lowered the quality of reconstruction for every algorithm, while the white and black noise distribution had a smaller but dataset-dependent effect. The L1-norm method showed greater stability and robustness, maintaining high clustering accuracy under high noise, and the other two methods were more sensitive to noise type and data structure.

Most robust was the L1-norm approach overall, which suggests that sparsity-based regularization effectively works to oppose the effects of noise. Future directions involve repeating these findings on more diverse datasets, developing hybrid NMF models that exploit the strengths of different regularizations, and exploring other clustering methods to further reveal the connection between reconstruction error and clustering accuracy.

6 Appendix

Instructions for running code

The submitted Jupyter notebook is intended to be run sequentially in one pass from start to finish. There should be no need to re-run previous cells, and doing so will also result in strange/inconsistent outputs as certain cells in the notebook will overwrite variable values from previous cells.

By default, the notebook assumes the ORL and YaleB datasets are located in a mounted Google Drive directory - this is because Google Colab was used to host the notebook and facilitate simultaneous contributions.

If the datasets are stored locally, the user can skip running the first 2 code cells. They will still need to run the 3rd code cell in order to initialise the function to load the datasets. Additionally, they will need to manually change the path to the ORL and YaleB datasets before calling the 'load_dataset()' function in the 4th code cell. Following this, no additional changes to the notebook will need to be made.

The following Python libraries were utilised:

- google (v2.0.3) - used to mount Google Drive
- os (built-in) - used to check file structure of the dataset directory, already present in default template
- math (built-in) - used for a single ceiling function call during a calculation
- numpy (v2.0.2) - used for array data structures, already present in default template
- PIL (v11.3.0) - used to load image data, already present in default template
- matplotlib (v3.10.0) - used to plot/draw images, already present in default template
- Collections (built-in) - used to create a Counter object, already present in default template
- sklearn.cluster (1.6.1) - used to perform clustering, already present in default template
- sklearn.metrics (1.6.1) - used to evaluate clustering performance, already present in default template

All of the NMF algorithm implementations have been completed using base Python and some numpy linear algebra and basic mathematical functions.

References

Use unnumbered third level heading for the references. Any choice of citation style is acceptable as long as you are consistent. It is permissible to reduce the font size to 'small' (9-point) when listing the references.

[1] Quora. "When Would You Choose L1-norm over L2-norm?". Available at: https://www.quora.com/When-would-you-chose-L1-norm-over-L2-norm.

[2] Quora. "What Are the Benefits and Drawbacks of L1-norm, L2-norm, and L∞-norm". Available at: https://www.quora.com/What-are-the-benefits-and-drawbacks-of-L1-norm-L2-norm-and-Lmax-norm.

[3] Wang, D., Liu, J.–X., Gao, Y.–L., Yu, J., Zheng, C.–H., & Xu, Y. (2016). An NMF-L , -Norm Constraint Method for Characteristic Gene Selection. *PLOS ONE*. Available at: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0158494.

[4] Jain, A. "Salt and Pepper Noise and How to Remove It in Machine Learning". Available at: https://medium.com/@abhishekjainindore24/salt-and-pepper-noise-and-how-to-remove-it-in-machine-learning-032d76b138a5.

[5] Al-Azzeh, J., Zahran, B., & Alqadi, Z. (2018). Salt and Pepper Noise: Effects and Removal. *JOIV: International Journal on Informatics Visualization*, 2(4), 253–258. Available at: https://joiv.org/index.php/joiv/article/viewFile/151/149#:~:text=Salt%2Dand%2Dpepper%20noise%20is,signal%2C%20caused%20by%20external%20disturbance..

[6] Al Qadi, Z.A.A., Khawatreh, S., Nader, J., & Khrisat, M. (2020). ZA Special Filter to Eliminate Salt and Pepper Noise from Gray and Color Images. *JOIV: International Journal on Informatics Visualization*, 4(1), 1–8. Available at: https://www.researchgate.net/figure/Salt-and-pepper-noise-with-ratio-025_fig1_343721713.

[7] Tavarez, A. (2019). The ORL Database for Training and Testing. Available at: https://www.kaggle.com/datasets/tavarez/the-orl-database-for-training-and-testing.

[8] Kirtas, M., Passalis, N., & Tefas, A. (2023). Multiplicative Update Rules for Accelerating Deep Learning Training and Increasing Robustness. Available at: https://arxiv.org/pdf/2307.07189.

[9] Ang, A. M. S. (2019). Non-negative Matrix Factorization and Multiplicative Update. Available at: https://angms.science/doc/NMF/nmf_0.pdf.
[10] Georghiades, A. S., Belhumeur, P. N., & Kriegman, D. J. (2001). From Few to Many: Illumination Cone Models for Face Recognition under Variable Lighting and Pose. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(6), 643–660. Available at: https://vision.ucsd.edu/datasets/extended-yale-face-database-b-b.
[11] Streeter, M. (2023). Universal Majorization-Minimization Algorithms. Available at: https://arxiv.org/pdf/2308.00190.
[12] Koshelev, I., & Lefkimmiatis, S. (2023). Iterative Reweighted Least Squares Networks With Convergence Guarantees for Solving Inverse Imaging Problems. Available at: https://arxiv.org/pdf/2308.05745.
[13] Mairal, J. (2015). Incremental Majorization-Minimization Optimization with Application to Large-Scale Machine Learning. Available at: https://arxiv.org/pdf/1402.4419.
[14] Streeter, M. (2023). Universal Majorization-Minimization Algorithms. Available at: https://arxiv.org/pdf/2308.00190.
[15] Milvus. "What Noise Distributions Are Most Commonly Used (e.g., Gaussian)". Available at: https://milvus.io/ai-quick-reference/what-noise-distributions-are-most-commonly-used-eg-gaussian.
[16] Bingham, E., & Mannila, H. (2001). Random projection in dimensionality reduction: Applications to image and text data. In Proceedings of the 7th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 245–250). ACM. https://doi.org/10.1145/2063576.2063676.

Contribution

Nathan Roland: Wrote all code in the YALE B dataset section, completed related work, and method sections of report

Leheng Li: Writing NMF algorithm code, complete introduction, abstract and conclusion, participate in method and related work part in the report.

Matthew Xu: Evaluation/experiment code, experiment results and discussion section in the report

Henry Nguyen: Evaluation/experiment code, experiment results and discussion section in the report